



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO COMPLEMENTOS DE BASES DE DATOS

Sistema Inteligente de Gestión Logística con ArangoDB

Realizado por

**Ismael Carrasco Mkhazni
Guillermo García León**

Grado en Ingeniería Informática - Ingeniería del Software

Resumen

Este proyecto presenta el diseño e implementación de una plataforma de gestión logística utilizando una base de datos multimodelo (ArangoDB). El sistema permite gestionar una red de almacenes y productos mediante grafos, optimizando el cálculo de rutas y el control de inventarios en tiempo real. La solución se despliega mediante una arquitectura contenedorizada con Docker y una interfaz reactiva en Streamlit.

Palabras clave: ArangoDB, Grafos, Logística, AQL, Docker, Streamlit.

Abstract

This project presents the design and implementation of an advanced logistics management platform using a multi-model database (ArangoDB). The system allows managing a network of warehouses and products using graphs, optimizing route calculation and real-time inventory control. The solution is deployed using a containerized architecture with Docker and a reactive interface in Streamlit.

Keywords: ArangoDB, Graphs, Logistics, AQL, Docker, Streamlit.

Índice general

1	Introducción	7
1.1.	Contexto y Motivación	7
1.2.	Definición del Problema	7
1.3.	Objetivos del Proyecto	7
1.4.	Metodología y Desafíos Técnicos	8
2	Estudio previo	9
2.1.	Introducción	9
2.2.	Justificación Tecnológica: ArangoDB	9
2.2.1.	Arquitectura Multimodelo Nativa	9
2.3.	Modelado de Grafos frente al Modelo Relacional	9
2.4.	Lenguaje de Consultas AQL	10
2.5.	Integración del Stack Tecnológico	10
3	Análisis del problema	11
3.1.	Introducción	11
3.2.	Requisitos de información	11
3.3.	Requisitos funcionales	11
3.4.	Requisitos no funcionales	12
4	Diseño de la solución	13
4.1.	Introducción	13
4.2.	Arquitectura del Grafo: RedLogistica	13
4.2.1.	Colecciones de Vértices (Nodos)	13
4.2.2.	Colecciones de Aristas (Edges)	13
4.3.	Diagrama del Modelo de Datos	13
4.4.	Estrategia de Integridad Referencial	14
5	Implementación	15
5.1.	Introducción	15
5.2.	Desarrollo del Backend: Persistencia y Lógica de Datos	15
5.2.1.	Capa de Conexión (app/database.py)	15
5.2.2.	Inicialización Automatizada (data/setup_data.py)	16
5.2.3.	Lógica de Negocio y Grafos (app/crud.py)	16
5.3.	Desarrollo del Frontend: Interfaz y Visualización	17
5.3.1.	Arquitectura y Gestión de Estado	17
5.3.2.	Módulos de Operación Logística	17
5.3.3.	Visualización Avanzada con <i>Vis.js</i>	17
5.4.	Lógica de Consultas en AQL (ArangoDB Query Language)	18
5.4.1.	Cálculo de Rutas Óptimas (Algoritmo de Dijkstra)	19
5.4.2.	Gestión Atómica de Stock (UPSERT)	19

5.4.3. Integridad Referencial Manual (Borrado en Cascada)	19
5.5. Infraestructura y Despliegue (docker-compose.yml)	20
6 Pruebas	21
6.1. Introducción	21
6.2. Validación del Algoritmo de Rutas	21
6.3. Simulación de Contingencias	21
6.4. Conclusiones	21
7 Manual de Despliegue	22
7.1. Prerrequisitos del Sistema	22
7.2. Despliegue del Sistema	22
8 Manual de Usuario	23
8.1. Acceso e Interfaz Principal	23
8.2. Módulo de Inventario	23
8.3. Módulo de Rutas	23
8.4. Módulo de Administración	24
9 Conclusiones	25
Bibliografía	26

Índice de figuras

4.1. Representación lógica del grafo multimodelo RedLogistica.	14
--	----

Índice de extractos de código

5.1.	Inyección de dependencias en database.py	15
5.2.	Firma de la función de cálculo de rutas en crud.py	16
5.3.	Integración de Vis.js para visualización de grafos	18
5.4.	Consulta AQL para el camino más corto	19
5.5.	Lógica de inventario atómico	19
5.6.	Borrado en cascada de un almacén	20
5.7.	Orquestación de servicios en docker-compose.yml	20
7.1.	Comando de despliegue	22

1. Introducción

1.1. Contexto y Motivación

En el marco de la asignatura de Complementos de Bases de Datos, este proyecto nace con el objetivo de abordar uno de los retos que se presentan en la ingeniería del software: la gestión eficiente de redes logísticas. La distribución de mercancías no solo requiere el almacenamiento de datos estáticos, sino el procesamiento ágil de relaciones complejas entre almacenes, rutas y existencias.

Tradicionalmente, este problema se ha abordado mediante modelos relacionales. Sin embargo, hemos identificado que el uso de grafos ofrece una ventaja competitiva superior para modelar la realidad física de una red de transporte. Al tratar los almacenes como nodos y las carreteras como aristas, nuestro sistema permite una adaptabilidad inmediata ante cambios en el entorno, optimizando la ejecución de consultas de rutas y stock. Además, los grafos permiten una representación eficiente de relaciones complejas, lo que, en sistemas relacionales, generaría una carga computacional excesiva debido a la recursividad.

1.2. Definición del Problema

El sistema diseñado debe dar solución a una red logística integral compuesta por almacenes distribuidos geográficamente y un catálogo maestro de productos. El desafío planteado no se limita únicamente a la persistencia de la información, sino que abarca tres pilares fundamentales:

- **Gestión de Conectividad:** El sistema debe modelar rutas físicas con distancias variables y ser capaz de calcular el camino más corto de forma dinámica.
- **Control de Inventario Cruzado:** Es necesario vincular productos con múltiples ubicaciones, permitiendo operaciones de entrada y salida de stock en tiempo real.
- **Interactividad y Administración:** Para que la solución sea funcional, debe presentar una interfaz intuitiva que permita a los administradores visualizar la red, modificar su topología y gestionar el stock sin necesidad de conocimientos técnicos en lenguajes de consulta.

1.3. Objetivos del Proyecto

El propósito de este trabajo es el diseño e implementación de una plataforma de gestión logística inteligente. Para alcanzar esta meta, nos hemos fijado los siguientes

objetivos estratégicos:

1. **Modelado de Datos Avanzado:** Diseñar un grafo multimodelo en ArangoDB que represente fielmente la infraestructura logística.
2. **Arquitectura Robusta:** Implementar un backend modular basado en el Patrón Repositorio que asegure la escalabilidad y mantenibilidad del código.
3. **Visualización Dinámica:** Desarrollar un dashboard interactivo utilizando Streamlit y bibliotecas de visualización de redes (Vis.js) para ofrecer una experiencia de usuario clara.
4. **Despliegue Unificado:** Orquestar todo el ecosistema mediante contenedores Docker, que abstraiga la complejidad de la instalación, facilitando la puesta en marcha del proyecto en cualquier entorno de manera transparente y sencilla.

1.4. Metodología y Desafíos Técnicos

El desarrollo se ha llevado a cabo mediante una metodología iterativa y colaborativa, permitiendo que la lógica de negocio del backend y la interfaz del frontend evolucionen de forma sincronizada. A lo largo del proceso, hemos superado retos técnicos significativos:

- **Integridad en Grafos:** La implementación de borrados en cascada manuales mediante consultas AQL para evitar inconsistencias en la base de datos.
- **Sincronización de Servicios:** La configuración de una red interna en Docker que permita una comunicación fluida entre el servidor de base de datos y la aplicación.
- **Visualización de Datos:** La traducción de estructuras de grafos complejas a mapas interactivos comprensibles para el usuario final.

Con esta base, el presente documento detalla en los siguientes capítulos el análisis, diseño y validación de una solución que integra lo mejor de las bases de datos de grafos con las tecnologías web modernas.

2. Estudio previo

2.1. Introducción

En este capítulo se analiza y justifica el stack tecnológico seleccionado para el desarrollo del sistema. El núcleo del proyecto es la gestión de una red logística, un problema que por su naturaleza requiere un tratamiento avanzado de las relaciones entre datos. Por ello, el estudio se centra en las ventajas de la tecnología multimodelo y el procesamiento de grafos frente a las soluciones relacionales convencionales.

2.2. Justificación Tecnológica: ArangoDB

ArangoDB es una base de datos NoSQL nativa multimodelo. Esta característica es el pilar técnico del proyecto, ya que nos permite trabajar con documentos, grafos y sistemas clave-valor utilizando un único motor central y un mismo lenguaje de consultas: *AQL (ArangoDB Query Language)*.

2.2.1. Arquitectura Multimodelo Nativa

En el desarrollo de aplicaciones complejas, habitualmente es necesario integrar distintos motores (como MongoDB para documentos y Neo4j para grafos). La ventaja de ArangoDB es que combina estos modelos en un solo núcleo:

- **Modelo de Documentos:** Lo utilizamos para almacenar la información detallada y jerárquica de los almacenes y los productos en formato JSON.
- **Modelo de Grafos:** Es el corazón de la aplicación. Nos permite definir relaciones de conectividad entre ciudades y de ubicación de inventario mediante aristas.
- **Eficiencia operativa:** Al tener un solo motor, se eliminan los problemas de consistencia entre diferentes bases de datos y se simplifica drásticamente el despliegue mediante contenedores.

2.3. Modelado de Grafos frente al Modelo Relacional

La decisión de no utilizar una base de datos SQL tradicional se basa en la eficiencia en el recorrido de redes. En un modelo relacional, encontrar una ruta entre dos almacenes implicaría realizar múltiples operaciones de unión (*JOINS*) o recursividad compleja (CTEs), cuyo rendimiento decae exponencialmente con la profundidad de la red.

En nuestro modelo de grafos, las relaciones son registros físicos que contienen punteros a los documentos que conectan. Esto permite que el trazado de rutas sea una operación directa e inmediata, independientemente del tamaño total del conjunto de datos.

2.4. Lenguaje de Consultas AQL

AQL es un lenguaje declarativo que permite realizar operaciones complejas sobre los tres modelos de datos. Para este proyecto, su potencia radica en las funciones nativas de grafos:

- **Shortest Path:** Permite calcular la ruta mínima entre dos nodos basándose en pesos (distancia en km).
- **Traversals:** Facilita el recorrido de la red para, por ejemplo, listar todos los productos contenidos en una rama específica de la logística.

2.5. Integración del Stack Tecnológico

Para que el motor de base de datos sea funcional, lo hemos integrado en un ecosistema que garantiza la interactividad y la facilidad de uso:

- **Backend (Python):** Actúa como puente entre la interfaz y la base de datos, implementando la lógica de negocio y la validación de datos.
- **Frontend (Streamlit):** Proporciona un panel de control interactivo donde los cambios en la base de datos se visualizan de forma inmediata a través de mapas y tablas dinámicas.
- **Docker:** Asegura que todo este entorno sea reproducible, configurando la red interna para que la aplicación y la base de datos se comuniquen de forma transparente.

3. Análisis del problema

3.1. Introducción

En este capítulo detallamos los requisitos fundamentales que el sistema logístico debe satisfacer. El análisis se ha abordado desde una perspectiva integral, considerando tanto la robustez necesaria en la persistencia y el cálculo de datos en el backend como la agilidad requerida en la interfaz de usuario para una gestión operativa real.

3.2. Requisitos de información

Para modelar fielmente la red logística, el sistema debe ser capaz de persistir y relacionar entidades que poseen atributos tanto técnicos como descriptivos para su visualización:

- **Almacenes (Nodos):** Además de su identificación y capacidad, deben incluir atributos de localización (como el nombre de la ciudad) que sirvan como etiquetas dentro de la representación visual del grafo y permitan al usuario identificar cada centro logístico en el dashboard interactivo.
- **Productos (Nodos):** Deben categorizarse mediante identificadores únicos (SKU), nombres descriptivos y categorías para facilitar su filtrado en el inventario.
- **Conexiones Viales (Aristas):** Deben definir la topología de la red, vinculando almacenes mediante el atributo de distancia, esencial para los algoritmos de optimización.
- **Existencias (Aristas):** Representan la relación dinámica entre productos y almacenes, manteniendo un conteo preciso de unidades disponibles en cada punto de la red.

3.3. Requisitos funcionales

Hemos definido las capacidades del sistema centrándonos en la utilidad para el administrador final, equilibrando la potencia de cálculo con la facilidad de uso:

- **Gestión de Red Interactiva:** El sistema debe permitir realizar operaciones CRUD sobre almacenes, productos y rutas. Estas acciones deben reflejarse inmediatamente tanto en la base de datos como en la representación visual de la red.

- **Cálculo de Rutas Críticas:** Implementar una lógica capaz de determinar el camino más corto entre dos puntos, garantizando que el algoritmo recorra exclusivamente las rutas de transporte válidas.
- **Dashboard de Inventario:** Proporcionar una visión clara del stock, permitiendo realizar ajustes de unidades y visualizar la distribución de productos a través de la red.
- **Visualización Dinámica de la Red:** La interfaz debe generar un grafo interactivo que ayude al usuario a comprender la topología de la red logística de un solo vistazo.
- **Resiliencia ante Contingencias:** Capacidad de recalcular rutas de forma automática si una conexión queda inhabilitada, asegurando la continuidad de la cadena de suministro.

3.4. Requisitos no funcionales

Para asegurar la calidad técnica y la facilidad de evaluación del proyecto, hemos establecido los siguientes criterios:

- **Portabilidad:** El sistema debe ser independiente al entorno de ejecución, garantizando su funcionamiento inmediato mediante el uso de contenedores.
- **Rendimiento en Relaciones:** La arquitectura debe permitir realizar consultas de vecindad y caminos cortos sin la penalización de rendimiento propia de los sistemas relacionales convencionales.
- **Persistencia y Consistencia:** Se debe asegurar que las operaciones sobre el grafo mantengan la integridad de la red, evitando nodos o aristas huérfanas tras una eliminación.

4. Diseño de la solución

4.1. Introducción

En este capítulo describimos la arquitectura lógica del sistema, centrándonos en cómo hemos aprovechado la flexibilidad del modelo de grafos para representar una red logística compleja. El diseño se fundamenta en la capacidad de ArangoDB para tratar las relaciones (rutas y stock) como entidades con identidad propia.

4.2. Arquitectura del Grafo: RedLogistica

Hemos diseñado un grafo denominado RedLogistica que unifica la infraestructura física de transporte con el inventario de mercancías. La organización se divide en dos tipos de colecciones:

4.2.1. Colecciones de Vértices (Nodos)

- **Almacenes:** Representan los centros logísticos. Cada nodo contiene documentos JSON con atributos como la ciudad, el nombre descriptivo y la capacidad máxima de almacenaje.
- **Productos:** Constituyen el catálogo maestro. Almacenan el identificador único (SKU), nombre y categoría del producto.

4.2.2. Colecciones de Aristas (Edges)

- **Rutas:** Definen la conectividad vial entre almacenes. Son aristas dirigidas o bidireccionales que portan el atributo `distancia_km`, crítico para los cálculos de optimización.
- **Stock_en:** Representan la presencia de un producto en un almacén. Almacenan el atributo `cantidad`, permitiendo una relación dinámica de muchos a muchos entre productos y ubicaciones.

4.3. Diagrama del Modelo de Datos

Para facilitar la comprensión de la topología de la red, hemos elaborado el siguiente diagrama mediante el paquete *TikZ*, representando la jerarquía y las conexiones del grafo:

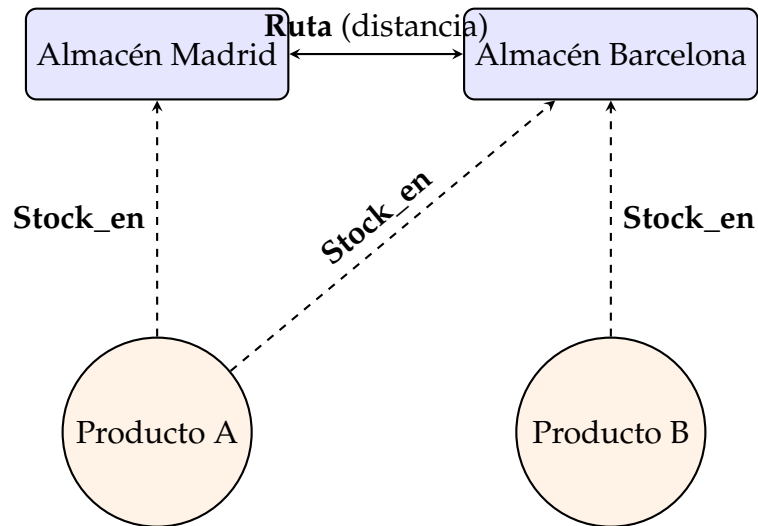


Figura 4.1: Representación lógica del grafo multimodelo RedLogística.

4.4. Estrategia de Integridad Referencial

Dada la naturaleza de las bases de datos NoSQL, hemos tomado la decisión de diseño de gestionar la integridad referencial desde la lógica de la aplicación. Para evitar “aristas huérfanas”, implementamos funciones en AQL que ejecutan borrados en cascada: al eliminar un nodo de Almacenes, el sistema identifica y elimina automáticamente todas las Rutas y registros de Stock_en asociados.

5. Implementación

5.1. Introducción

En este capítulo detallamos la construcción técnica del sistema. Hemos diseñado una arquitectura donde el backend garantiza la integridad de los grafos y el frontend ofrece una ventana intuitiva para la gestión operativa. Para asegurar un código escalable y mantenible, la lógica se ha estructurado siguiendo el Patrón Repositorio, separando estrictamente la capa de conexión, los módulos de persistencia (CRUD) y las vistas de usuario. Todo el sistema reside en un ecosistema contenedorizado que asegura su portabilidad.

5.2. Desarrollo del Backend: Persistencia y Lógica de Datos

El backend constituye el núcleo de inteligencia del sistema, encargado de traducir las reglas de negocio logísticas en operaciones eficientes sobre el motor ArangoDB.

5.2.1. Capa de Conexión (app/database.py)

La persistencia se centraliza en la clase `GestorLogisticaDB`, que actúa como punto de entrada único a la base de datos. Esta clase no solo instancia el cliente ArangoDB, sino que inicializa de forma segura las credenciales (usuario, contraseña y URL del host) mediante variables de entorno cargadas desde un archivo `.env`.

Un aspecto clave es el uso de la inyección de dependencias: al llamar al método `conectar()`, la instancia activa de la base de datos se vincula a cada uno de los repositorios especializados (almacenes, productos, rutas y stock).

```
1 def conectar(self):
2     """Establece la conexion a la base de datos y enlaza las clases CRUD.
3     """
4
5     self.db = self.client.db(self.db_name, username=self.username,
6                             password=self.password)
7
8     # Inyeccion de dependencias (Repository Pattern)
9     self.almacenes = CRUDAlmacenes(self.db)
10    self.rutas = CRUDRutas(self.db)
11    self.productos = CRUDProductos(self.db)
```


9 self.stock = CRUDStock(self.db)

Extracto de código 5.1: Inyección de dependencias en database.py

5.2.2. Inicialización Automatizada (data/setup_data.py)

Para inicializar la base de datos de manera segura, se ha desarrollado el script `setup_data.py`. Dado que en entornos de red aislados (como Docker) el motor de base de datos puede tardar más en arrancar que la aplicación, este script implementa un mecanismo de resiliencia con reintentos automáticos. Una vez establecida la conexión, ejecuta los métodos `inicializar_base_datos()` y `cargar_datos_prueba()`, creando las colecciones de vértices y aristas y poblando el grafo RedLogística con una infraestructura inicial de cinco ciudades y un catálogo maestro de productos.

5.2.3. Lógica de Negocio y Grafos (app/crud.py)

En este módulo se explotan las capacidades del lenguaje AQL (*ArangoDB Query Language*) para resolver problemas complejos de redes que en bases de datos relacionales serían computacionalmente costosos.

- **Gestión de Integridad en Cascada:** En bases de datos NoSQL de grafos, eliminar un nodo puede dejar aristas huérfanas. El método `eliminar` de `CRUDAlmacenes` resuelve esto mediante una consulta AQL transaccional que identifica y borra primero todas las Rutas y registros de `Stock_en` asociados al ID del almacén antes de eliminar el nodo físico.
- **Resolución de Caminos Mínimos:** La función `leer_optima` encapsula el algoritmo de Dijkstra nativo de ArangoDB. Se ha configurado explícitamente el parámetro `weightAttribute: 'distancia_km'` para asegurar que el cálculo se base en la longitud física de las carreteras y no simplemente en el número de saltos entre nodos.
- **Operaciones Atómicas de Stock (UPSERT):** La gestión del inventario utiliza una sentencia UPSERT en AQL. Esto permite que si la relación entre un producto y un almacén no existe, se cree (INSERT), y si ya existe, se actualice la cantidad (UPDATE) de forma atómica. Además, se ha integrado lógica condicional para asegurar que el stock nunca sea inferior a cero.

```
1 def leer_optima(self, origen, destino):
2     query = "FOR v, e IN ANY SHORTEST_PATH @o TO @d Rutas ..."
3     bind_vars = {'o': f'Almacenes/{origen.lower()}', 'd': f'Almacenes/{
4         destino.lower()}'}
```

Extracto de código 5.2: Firma de la función de cálculo de rutas en crud.py

5.3. Desarrollo del Frontend: Interfaz y Visualización

La interfaz de usuario, centralizada en `app/main.py`, ha sido diseñada bajo un paradigma de desarrollo reactivo utilizando el framework *Streamlit*. El objetivo principal es abstraer la complejidad de las consultas AQL, permitiendo que un operario logístico gestione la red sin conocimientos técnicos previos de la base de datos.

5.3.1. Arquitectura y Gestión de Estado

A diferencia de las aplicaciones web tradicionales que requieren un manejo complejo del DOM, *Streamlit* permite definir la interfaz de forma declarativa en Python. Para optimizar el rendimiento y la experiencia de usuario, se han implementado las siguientes estrategias:

- **Persistencia de Conexión:** Se utiliza el decorador `@st.cache_resource` para la función `inicializar_sistema()`. Esto asegura que la instancia del `GestorLogisticaDB` se mantenga en memoria durante toda la sesión del usuario, evitando el coste computacional de reconectar a `ArangoDB` en cada interacción.
- **Navegación Modular:** La aplicación se organiza mediante un menú lateral (*Sidebar*) que divide la lógica en tres pilares: Inventario, Rutas y Administración.

5.3.2. Módulos de Operación Logística

El dashboard implementa una lógica de pestañas (`st.tabs`) para organizar las funcionalidades sin saturar visualmente al usuario:

- **Gestión de Inventario:** Este módulo permite realizar cruces de información dinámicos. Por ejemplo, al consultar un almacén, el sistema ejecuta una trayectoria en el grafo para listar todos los productos vinculados a través de la arista `Stock_en`, presentando los resultados mediante tablas interactivas generadas con *Pandas*.
- **Administración Dinámica:** Se han diseñado formularios específicos (`st.form`) para realizar operaciones CRUD. Un aspecto destacado es la validación en tiempo real: el sistema impide, por ejemplo, crear rutas donde el origen y el destino sean la misma ciudad antes de enviar la petición al backend.

5.3.3. Visualización Avanzada con *Vis.js*

El componente más innovador del frontend es el Mapa de Red Interactivo. Dado que *Streamlit* no posee un componente nativo para grafos complejos, se ha

desarrollado una integración personalizada con la librería de JavaScript *Vis.js*.

El proceso de renderizado sigue este flujo técnico:

1. **Extracción y Formateo:** La aplicación extrae los documentos de la colección Almacenes (nodos) y Rutas (aristas). Los IDs internos de ArangoDB (ej. Almacenes/madrid) se procesan para mostrar nombres legibles.
2. **Serialización JSON:** Los datos se convierten a estructuras JSON que *Vis.js* puede interpretar, asignando propiedades visuales como el color de los nodos o el tipo de flecha en las aristas.
3. **Inyección de Componente:** Mediante `components.html`, se inyecta un entorno de ejecución de JavaScript que inicializa una instancia de `vis.Network`.

Esta integración permite al usuario interactuar físicamente con la red: los nodos responden a fuerzas de gravedad y tensión (*physics engine*), y las rutas muestran etiquetas de distancia en kilómetros directamente sobre las conexiones, facilitando la comprensión de la topología logística de un solo vistazo.

```
1 # Fragmento simplificado de la logica en main.py
2 html_code = f"""
3 <div id="mynetwork"></div>
4 <script>
5   var nodes = new vis.DataSet({json.dumps(nodos_lista)});
6   var edges = new vis.DataSet({json.dumps(aristas_lista)});
7   var options = {{
8     physics: {{ stabilization: false, barnesHut: {{ springLength: 200 }}
9     }},
10    edges: {{ smooth: {{ type: 'continuous' }} }}
11  }};
12  new vis.Network(container, data, options);
13 </script>
14 """
15 components.html(html_code, height=600)
```

Extracto de código 5.3: Integración de *Vis.js* para visualización de grafos

5.4. Lógica de Consultas en AQL (ArangoDB Query Language)

El valor diferencial de este proyecto reside en el uso de AQL para resolver problemas lógicos directamente en el motor de la base de datos. A continuación, se detallan las consultas más complejas implementadas en el sistema.

5.4.1. Cálculo de Rutas Óptimas (Algoritmo de Dijkstra)

Para encontrar el camino más corto entre dos centros logísticos, se utiliza la función nativa `SHORTEST_PATH`. A diferencia de una base de datos relacional, donde esto requeriría múltiples operaciones de unión (*JOINS*) o recursividad compleja, en ArangoDB se resuelve de forma declarativa.

```
1 LET ruta = (  
2   FOR v, e IN ANY SHORTEST_PATH @origen TO @destino Rutas  
3   OPTIONS { weightAttribute: 'distancia_km' }  
4   RETURN {  
5     ciudad: v.ciudad,  
6     distancia_tramo: e ? e.distancia_km : 0  
7   }  
8 )  
9 FILTER LENGTH(ruta) > 0  
10 RETURN {  
11   ciudades_ruta: ruta[*].ciudad,  
12   distancia_total_km: SUM(ruta[*].distancia_tramo),  
13   detalle_tramos: ruta  
14 }
```

Extracto de código 5.4: Consulta AQL para el camino más corto

5.4.2. Gestión Atómica de Stock (UPSERT)

La actualización de inventario debe ser robusta ante la concurrencia. Se ha implementado una lógica UPSERT que decide si insertar una nueva relación de stock o actualizar la existente, asegurando además que el inventario nunca sea negativo.

```
1 UPSERT { _from: @producto, _to: @almacen }  
2 INSERT {  
3   _from: @producto,  
4   _to: @almacen,  
5   cantidad: @variacion > 0 ? @variacion : 0  
6 }  
7 UPDATE {  
8   cantidad: (OLD.cantidad + @variacion) < 0 ? 0 : (OLD.cantidad +  
9   @variacion)  
9 } IN Stock_en
```

Extracto de código 5.5: Lógica de inventario atómico

5.4.3. Integridad Referencial Manual (Borrado en Cascada)

Dado que ArangoDB no impone integridad referencial rígida en las aristas por defecto, se ha diseñado una consulta que limpia todas las dependencias antes de

eliminar un nodo de tipo almacén.

```
1 LET aristas_rutas = (  
2   FOR r IN Rutas  
3   FILTER r._from == @id_almacen OR r._to == @id_almacen  
4   REMOVE r IN Rutas  
5 )  
6 LET aristas_stock = (  
7   FOR s IN Stock_en  
8   FILTER s._to == @id_almacen  
9   REMOVE s IN Stock_en  
10 )  
11 REMOVE @key_almacen IN Almacenes
```

Extracto de código 5.6: Borrado en cascada de un almacen

5.5. Infraestructura y Despliegue (docker-compose.yml)

Para asegurar la portabilidad absoluta, se ha orquestado el despliegue mediante contenedores Docker.

- **Dockerfile:** Basado en una imagen ligera de python:3.10-slim para minimizar el tamaño del contenedor, instalando solo las dependencias estrictamente necesarias como python-arango y streamlit.
- **Orquestación de Servicios:** El archivo docker-compose.yml define una red aislada donde el servicio db (ArangoDB) y el servicio app (la aplicación) se comunican de forma transparente.

Un detalle técnico relevante es la instrucción `command` en el servicio de la aplicación. Se utiliza una cadena de comandos en Bash para asegurar que, antes de lanzar la interfaz de Streamlit, se ejecute con éxito el script de configuración de datos, garantizando que el usuario siempre encuentre una base de datos funcional al acceder al sistema.

```
1 services:  
2   db:  
3     image: arangodb:latest  
4     environment:  
5       - ARANGO_ROOT_PASSWORD=password_seguro  
6   app:  
7     build: .  
8     depends_on:  
9       - db  
10    environment:  
11      - ARANGO_URL=http://db:8529  
12      - PYTHONPATH=/app  
13    command: bash -c "python data/setup_data.py && streamlit run app/main.py"
```

Extracto de código 5.7: Orquestación de servicios en docker-compose.yml

6. Pruebas

6.1. Introducción

Las pruebas del sistema se han enfocado en validar la exactitud algorítmica en el recorrido de grafos y en comprobar la capacidad dinámica del sistema frente a alteraciones inesperadas en la topología de la red.

6.2. Validación del Algoritmo de Rutas

Durante la evaluación de las consultas AQL, detectamos una anomalía semántica importante: el algoritmo `SHORTEST_PATH` nativo tendía a utilizar los nodos de Productos como atajos entre ciudades, dado que estas aristas no contaban con atributo de distancia. Esto devolvía rutas carentes de sentido físico (ej. 350 km entre Madrid y Valencia saltando a través de un producto).

Logramos resolver este comportamiento restringiendo explícitamente el motor AQL para que evaluara de manera exclusiva la colección de aristas Rutas, obligando al algoritmo de Dijkstra interno a considerar únicamente carreteras reales. Tras aplicar esta restricción, las métricas fueron exactas.

6.3. Simulación de Contingencias

Para verificar la resiliencia del modelo, desarrollamos un test de estrés eliminando deliberadamente la arista principal que conectaba Madrid y Barcelona (simulando un accidente o corte vial). Al volver a solicitar el recálculo de la ruta óptima, el sistema reaccionó con una adaptabilidad inmediata: redirigió el flujo logístico a través de nodos intermedios alternativos (vía Bilbao) y ajustó la distancia total de 620 km a 1010 km de manera totalmente transparente, sin requerir una sola modificación en la lógica de backend.

6.4. Conclusiones

Las pruebas confirman el éxito rotundo del modelo orientado a grafos. El sistema reacciona en tiempo real a los cambios estructurales, demostrando que ArangoDB es una solución técnica superior para la planificación logística.

7. Manual de Despliegue

7.1. Prerrequisitos del Sistema

Para garantizar la portabilidad absoluta y evitar conflictos de dependencias locales, el sistema ha sido completamente contenedorizado. Por tanto, el único requisito previo para la ejecución del proyecto es contar con las siguientes herramientas instaladas en la máquina host:

- **Docker:** Motor de contenedores.
- **Docker Compose (V2):** Herramienta de orquestación (se recomienda usar la versión integrada en Docker Desktop o la V2 para evitar errores de compatibilidad).

7.2. Despliegue del Sistema

Gracias a la orquestación configurada, el levantamiento de toda la infraestructura requiere un único comando.

1. Abra una terminal y navegue hasta el directorio raíz del proyecto (donde se ubica el archivo `docker-compose.yml`).
2. Ejecute el siguiente comando para construir las imágenes y levantar los servicios:

```
1 docker compose up --build
```

Extracto de código 7.1: Comando de despliegue

Al ejecutar el comando anterior, Docker levantará dos servicios simultáneamente: la base de datos (db) y la aplicación (app). Durante este proceso, el servicio de la aplicación ejecutará internamente el script `data/setup_data.py` antes de iniciar la interfaz. Este script esperará a que el motor de ArangoDB esté completamente listo y, a continuación, inicializará las colecciones, el grafo lógico y poblará la base de datos con los datos de prueba. La página se encontrará disponible en la URL: `http://localhost:8501`

8. Manual de Usuario

8.1. Acceso e Interfaz Principal

La plataforma cuenta con una interfaz web reactiva diseñada para abstraer la complejidad de las consultas a la base de datos de grafos. Al acceder al sistema (por defecto en `http://localhost:8501`), el operario logístico encontrará un diseño limpio cuyo elemento principal de navegación es un menú lateral. Este menú permite alternar entre las tres áreas funcionales del sistema: *Inventario*, *Rutas* y *Administración*.

8.2. Módulo de Inventario

Este módulo está dedicado a la supervisión y control del stock. Se divide en tres pestañas operativas:

- **Ver por Almacén:** Permite al usuario seleccionar un almacén (Ciudad) desplegando una tabla dinámica con todos los productos vinculados a través de la arista `Stock_en`.
- **Ver por Producto:** Facilita la búsqueda inversa. Al seleccionar el identificador de un producto, el sistema rastrea y muestra una lista de todos los almacenes de la red que disponen de existencias.
- **Modificar Stock:** Interfaz transaccional para registrar entradas o salidas de mercancía mediante sentencias UPSERT en AQL. El usuario introduce la variación de unidades (positivo para añadir, negativo para retirar). El sistema garantiza que el stock nunca sea inferior a cero.

8.3. Módulo de Rutas

Diseñado para la visualización y optimización del transporte, este módulo consta de dos herramientas clave:

- **Cálculo de Rutas Óptimas:** Herramienta de toma de decisiones que implementa el algoritmo de Dijkstra nativo de ArangoDB. Al calcular la ruta entre un origen y un destino, el sistema considera el peso físico de las carreteras (`distancia_km`) y devuelve el detalle exacto de los tramos.
- **Mapa de Red Interactivo:** Un lienzo visual interactivo desarrollado con la librería `Vis.js`. El usuario puede interactuar físicamente con la red: los nodos

responden a fuerzas de gravedad y las rutas muestran etiquetas de distancia directamente sobre las conexiones.

8.4. Módulo de Administración

Sección reservada para alterar la topología de la cadena de suministro mediante formularios validados:

- **Almacenes:** Permite expandir la red registrando nuevos centros logísticos con su capacidad. Para mantener la integridad, la eliminación de un almacén ejecuta un borrado en cascada que retira automáticamente todas las rutas y registros de stock asociados.
- **Productos:** Formulario para gestionar el catálogo maestro, permitiendo crear artículos definiendo su identificador (SKU), nombre y categoría.
- **Rutas:** Permite construir nuevas conexiones viales o actualizar las existentes definiendo la distancia entre ciudades. El sistema incluye validaciones en tiempo real para impedir errores lógicos, como crear rutas con el mismo origen y destino.

9. Conclusiones

El desarrollo integral de este sistema logístico ha servido para validar de forma empírica la idoneidad de las bases de datos de grafos para la resolución de problemas de enrutamiento que, históricamente, penalizan severamente el rendimiento en sistemas de naturaleza relacional o tabular.

El uso de un diseño multimodelo en ArangoDB ha simplificado drásticamente el modelo mental del dominio de negocio. Paralelamente, la adopción del Patrón Repositorio en la capa de software ha asegurado un código legible y escalable. La contenedorización completa del ciclo de vida del proyecto mediante Docker me ha permitido ofrecer una solución “llave en mano”, mitigando la clásica problemática de las dependencias locales y garantizando un despliegue transparente para su evaluación.

Aunque la curva de aprendizaje inicial del lenguaje AQL y del razonamiento transversal en grafos fue pronunciada, los resultados técnicos conseguidos demuestran una capacidad resolutive sólida y justifican plenamente la elección tecnológica, logrando una plataforma altamente tolerante a fallos logísticos.

Uso de Herramientas de IA

En cumplimiento con las directrices de la asignatura “Complementos de Bases de Datos”, declaramos el uso de modelos generativos de IA (Gemini, de Google) como herramienta de apoyo técnico durante el desarrollo del proyecto. Sus usos específicos han sido:

- **Análisis Arquitectónico y Refactorización:** Soporte consultivo en la transición del código de un modelo centralizado a uno distribuido (Patrón Repositorio) para la separación de responsabilidades en la base de datos.
- **Depuración de Despliegue:** Asistencia técnica puntual en la comprensión de jerarquías de directorios dentro de Docker para resolver el fallo de importación circular (PYTHONPATH).
- **Formato y Estilo:** Apoyo en la adaptación y formateo de la documentación técnica al lenguaje de marcado LaTeX.

Como autores de la entrega, declaramos que todo el código fuente y el contenido teórico generado ha sido analizado de manera crítica, depurado y validado empíricamente a través de nuestras pruebas. Comprendemos en profundidad cada bloque lógico y componente del sistema expuesto en esta memoria.

Bibliografía
